Histórias e Critérios de aceite

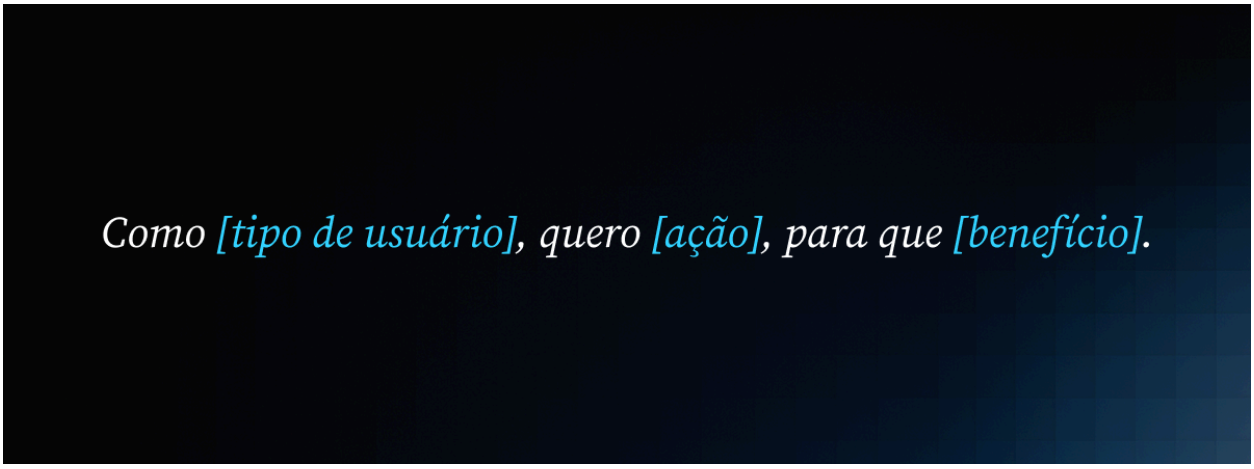
Na seção anterior, você viu que requisitos são decisões sobre como a solução deve funcionar. Mas decisões escritas de forma técnica têm um problema: elas chegam ao time de desenvolvimento sem o contexto humano que as originou. Quem vai usar isso? Por que precisa? O que muda na vida de alguém quando essa funcionalidade existir?

É aí que entram as histórias de usuário.

O que é uma história de usuário

Uma história de usuário não é uma forma diferente de escrever um requisito. É uma tradução do requisito para o ponto de vista de quem vai usar a solução. Ela descreve quem precisa, o que precisa fazer e por que aquilo tem valor. O objetivo é humanizar a necessidade antes que o time vá construir qualquer coisa.

A estrutura é simples e deliberadamente enxuta:



"Como [tipo de usuário], quero [ação], para que [benefício]."

Dois exemplos concretos:

- "Como cliente, quero receber um lembrete antes do vencimento da fatura para não esquecer de pagar."

- "Como atendente, quero visualizar o histórico de pedidos do cliente para responder dúvidas sem precisar pedir que ele repita as informações."

Perceba que nenhuma das duas frases menciona botão, tela, API ou banco de dados. A história existe para alinhar o *porquê* antes de qualquer decisão de *como*. O detalhe técnico vem depois, na conversa de refinamento, não dentro da história em si.

Um erro comum é começar a escrever histórias muito cedo, ainda na fase de levantamento inicial. Quando o escopo macro ainda não está claro, o excesso de histórias detalhadas cria a falsa impressão de que o projeto avançou, quando na verdade o que aconteceu foi que uma área foi especificada antes de entender o todo. A história é o passo final da especificação, não o primeiro.

O que são critérios de aceite

Se a história de usuário diz o *quê* e o *porquê*, os critérios de aceite dizem *quando aquilo está pronto de verdade*.

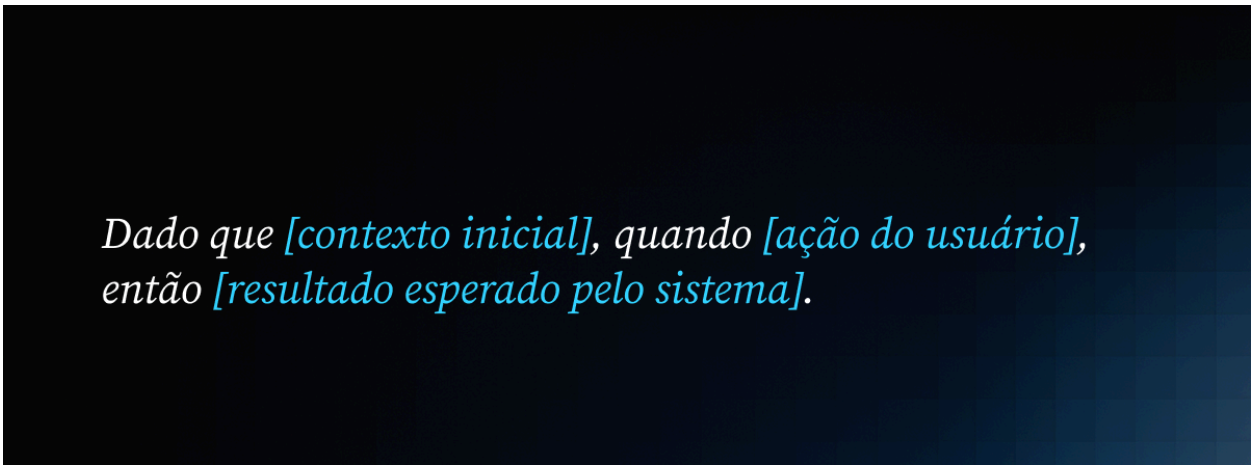
Critérios de aceite são as condições que precisam ser verdadeiras para que uma história seja considerada concluída. Eles formam o contrato entre quem pediu e quem construiu. Sem eles, "pronto" é uma opinião. Com eles, "pronto" é um fato verificável.

Bons critérios de aceite são concretos, testáveis e escritos antes do desenvolvimento começar. Veja a diferença na prática:

- **Critério vago:** "O sistema deve exibir os pedidos do cliente."
- **Critério útil:** "Ao acessar o histórico, o atendente visualiza os últimos 12 meses de pedidos, ordenados do mais recente para o mais antigo, com status atualizado em tempo real."

A segunda versão não deixa margem para interpretação. O dev sabe exatamente o que implementar. O QA sabe exatamente o que testar. E você, designer, sabe exatamente o que precisa estar na interface.

Uma técnica muito usada para escrever critérios de aceite de forma estruturada é o formato BDD (Behavior Driven Development), com a estrutura *Given / When / Then*:



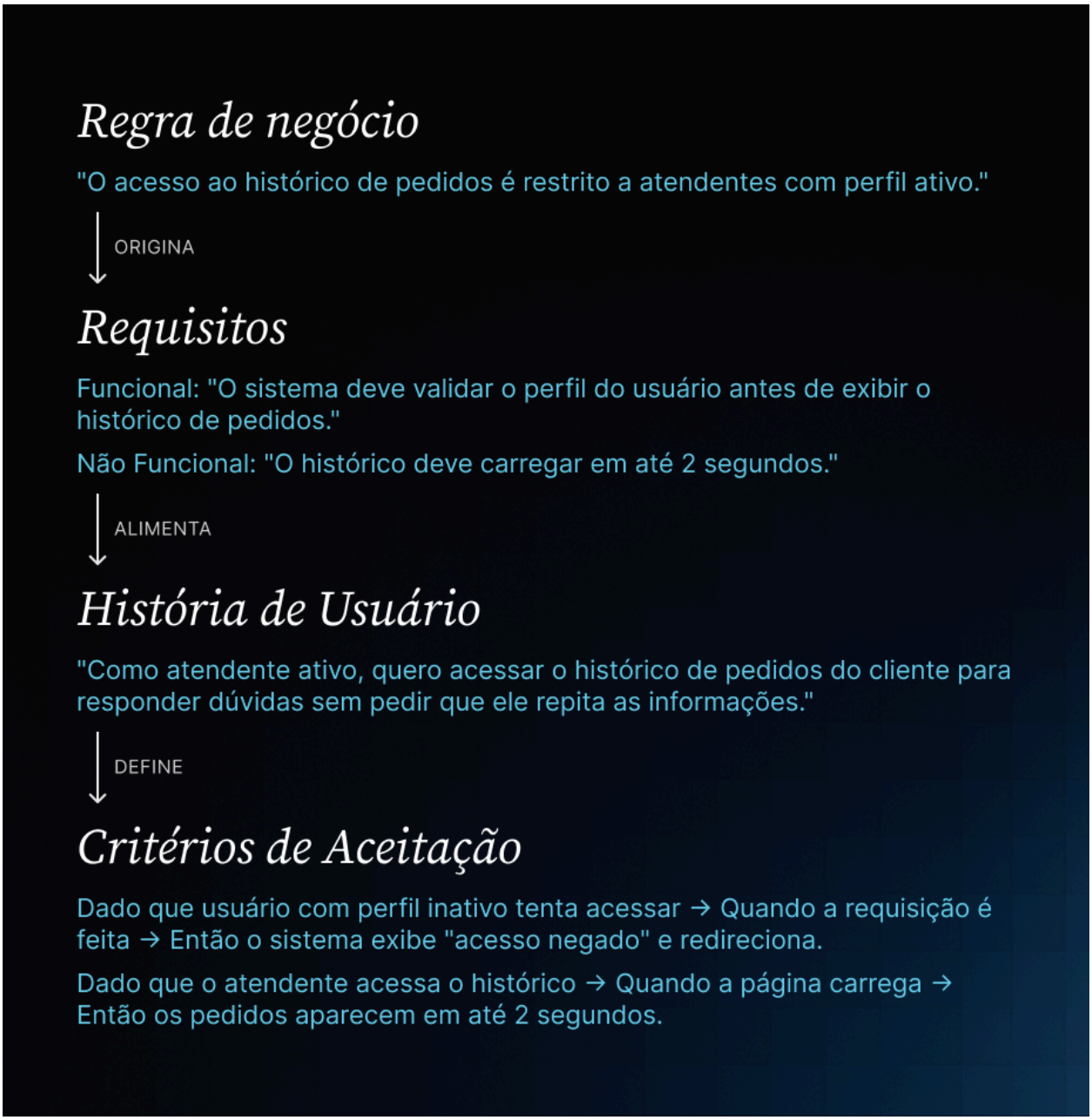
Dado que [contexto inicial], **quando** [ação do usuário], **então** [resultado esperado pelo sistema].

Exemplo: *Dado que* o atendente está na tela de histórico de um cliente, *quando* ele seleciona o filtro "últimos 3 meses", *então* o sistema exibe apenas os pedidos dentro desse período, atualizando a listagem sem recarregar a página.

Esse formato força quem escreve a pensar no comportamento completo, não apenas no caminho feliz. E é exatamente nos caminhos não felizes que mora a maioria dos requisitos esquecidos.

Como tudo isso se relaciona

Agora que os quatro conceitos estão definidos, vale ver como eles se encaixam na prática. Eles não são peças independentes — cada um deriva do anterior e o complementa.



Tudo começa nas **regras de negócio**, que existem independentemente de qualquer sistema e definem o que o negócio exige. A partir delas, surgem os **requisitos funcionais e não funcionais**, que traduzem essas exigências em comportamentos concretos que a solução precisa ter. Os requisitos funcionais alimentam as **histórias de usuário**, que colocam esses comportamentos na perspectiva de quem vai usar. E cada história carrega seus **critérios de aceite**, que definem quando aquele comportamento foi entregue corretamente — e onde

os requisitos não funcionais, como performance e segurança, finalmente ganham forma testável.

Um exemplo completo, com o mesmo contexto percorrendo toda a cadeia:

Regra de negócio: "O acesso ao histórico de pedidos é restrito a atendentes com perfil ativo."

Requisito funcional: "O sistema deve validar o perfil do usuário antes de exibir o histórico de pedidos."

Requisito não funcional: "O histórico deve carregar em até 2 segundos."

História de usuário: "Como atendente ativo, quero acessar o histórico de pedidos do cliente para responder dúvidas sem pedir que ele repita as informações."

Critérios de aceite:

- *Dado que* um usuário com perfil inativo tenta acessar o histórico, *quando* a requisição é feita, *então* o sistema exibe mensagem de acesso negado e redireciona para a tela inicial.
- *Dado que* o atendente acessa o histórico de um cliente, *quando* a página carrega, *então* os pedidos são exibidos em até 2 segundos.

Cada nível responde a uma pergunta diferente. A regra responde *o que o negócio permite*. Os requisitos respondem *o que o sistema precisa fazer e com qual qualidade*. A história responde *quem precisa e por quê*. Os critérios respondem *como saber que está correto*.

Vale reforçar um ponto que costuma gerar confusão: requisitos não viram histórias numa relação de um para um. Um único requisito funcional pode gerar várias histórias quando representa uma funcionalidade complexa. E um requisito não funcional raramente vira uma história própria — ele entra como critério de aceite da história à qual pertence, tornando a régua de qualidade visível e testável para o time.

Com esse vocabulário completo em mãos, o próximo passo é colocar a mão na massa. Saber o que são regras, requisitos, histórias e critérios é necessário, mas não suficiente. A dúvida que aparece na prática é outra: o que, de fato, precisa estar escrito para que uma especificação seja útil? Nem de mais, nem de menos. É exatamente isso que veremos na próxima seção.